

Traversal in Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list
```

```
class ListNode{
```

```
public:
```

```
int data;
```

```
ListNode *next;
```

```
ListNode() : data(0), next(nullptr) {}
```

```
ListNode(int x) : data(x), next(nullptr) {}
```

```
ListNode(int x, ListNode *next) : data(x),  
next(next) {}
```

```
};
```

```
class Solution {
```

```
public:
```

```
//Function for Linked List Traversal
```

```
vector<int> LLTraversal(ListNode* head) {
```

```
    //Storing a copy of the linked list
```

```
    ListNode* temp = head;
```

```
    //To store the values
```

```
//Sequentially  
vector<int> ans;  
  
//Keep traversing  
//Until the nullptr  
//Is not encountered  
while (temp != nullptr) {  
    //Storing of the values  
    ans.push_back(temp->data);  
    //Storing the address of the next node  
    temp = temp->next;  
}  
  
//Return answer  
return ans;  
}  
};
```

```
int main() {  
    //Manual creation of nodes  
    ListNode* y1 = new ListNode(2);  
    ListNode* y2 = new ListNode(5);
```

```
ListNode* y3 = new ListNode(8);
```

```
ListNode* y4 = new ListNode(7);
```

```
// Linking the nodes
```

```
y1->next = y2;
```

```
y2->next = y3;
```

```
y3->next = y4;
```

```
//Instance of
```

```
//Solution class
```

```
Solution sol;
```

```
//Calling LLTraversal method
```

```
//To get the values
```

```
vector<int> result = sol.LLTraversal(y1);
```

```
//Printing the result
```

```
cout << "Linked List Values:" << endl;
```

```
for (int val : result) {
```

```
    cout << val << " ";
```

```
}
```

```
cout << endl;
```

```
//Clean up
```

```
//Allocated memory
```

```
delete y1;
```

```
delete y2;
```

```
delete y3;
```

```
delete y4;
```

```
return 0;
```

```
}
```

Deletion of the head of LL

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Node structure
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode(): val(0), next(nullptr) {}
```

```
    ListNode(int data1): val(data1), next(nullptr) {}
```

```
ListNode(int data1, ListNode *next1): val(data1),  
next(next1) {}  
};
```

```
class Solution {
```

```
public:
```

```
// Function to delete the head node of the linked  
list
```

```
ListNode* deleteHead(ListNode* head) {
```

```
// If list is empty, nothing to delete
```

```
if (head == nullptr)
```

```
    return nullptr;
```

```
// Set temporary pointer
```

```
ListNode* temp = head;
```

```
// Update head to the next node
```

```
head = head->next;
```

```
// Delete original head
```

```
delete temp;
```

```
    // Return new head  
    return head;  
}  
};
```

// Function to print the linked list

```
void printList(ListNode* head) {  
    ListNode* current = head;  
    while (current != nullptr) {  
        cout << current->val << " ";  
        current = current->next;  
    }  
    cout << endl;  
}
```

// Function to insert a new node at the beginning of the linked list

```
ListNode* insertAtHead(ListNode* head, int data) {  
    ListNode* newNode = new ListNode(data);  
    newNode->next = head;  
    head = newNode;  
    return head;
```

}

int main() {

// Create a linked list

ListNode* head = nullptr;

head = insertAtHead(head, 3);

head = insertAtHead(head, 2);

head = insertAtHead(head, 1);

cout << "Original list: ";

printList(head);

// Creating an instance of Solution Class

Solution sol;

// Function call to delete the head node

head = sol.deleteHead(head);

cout << "List after deleting head: ";

printList(head);

```
    return 0;
}
```

Deletion of the tail of Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Node structure
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode(): val(0), next(nullptr) {}
```

```
    ListNode(int data1): val(data1), next(nullptr) {}
```

```
    ListNode(int data1, ListNode *next1): val(data1),  
next(next1) {}
```

```
};
```

```
class Solution {
```

```
public:
```

```
    // Function to delete the tail node of linked list
```

```
    ListNode* deleteTail(ListNode* head) {
```

```
        // If the list is empty or has only one node
```



```
if (head == NULL || head->next == NULL)  
    return NULL; // Return NULL
```

```
// Temporary pointer
```

```
ListNode* temp = head;
```

```
/*Traverse to the second last  
node in the list*/
```

```
while (temp->next->next != NULL) {  
    temp = temp->next;  
}
```

```
// Delete the last node
```

```
delete temp->next;
```

```
/*Set the next of the second  
last node to nullptr,  
effectively removing the last node*/
```

```
temp->next = nullptr;
```

```
// Return head of modified list
```

```
        return head;
    }
};
```

// Function to print the linked list

```
void printList(ListNode* head) {
    ListNode* current = head;
    while (current != nullptr) {
        cout << current->val << " ";
        current = current->next;
    }
    cout << endl;
}
```

// Function to insert a new node at the beginning of the linked list

```
ListNode* insertAtHead(ListNode* head, int data) {
    ListNode* newNode = new ListNode(data);
    newNode->next = head;
    head = newNode;
    return head;
}
```

```
int main() {  
  
    // Create a linked list  
  
    ListNode* head = nullptr;  
  
    head = insertAtHead(head, 3);  
  
    head = insertAtHead(head, 2);  
  
    head = insertAtHead(head, 1);  
  
  
    cout << "Original list: ";  
  
    printList(head);  
  
  
    // Creating an instance of Solution Class  
  
    Solution sol;  
  
  
    // Function call to delete the tail node  
  
    head = sol.deleteTail(head);  
  
  
    cout << "List after deleting head: ";  
  
    printList(head);  
  
  
    return 0;
```

```
}
```

Deletion of the Kth element of Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Node structure
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode(): val(0), next(nullptr) {}
```

```
    ListNode(int data1): val(data1), next(nullptr) {}
```

```
    ListNode(int data1, ListNode *next1): val(data1),  
next(next1) {}
```

```
};
```

```
class Solution {
```

```
public:
```

```
    // Function to delete the k-th node of a linked list
```

```
    ListNode* deleteKthNode(ListNode* head, int k)  
{
```

```
        // If the list is empty, return NULL
```

```
        if (head == NULL)
```

return NULL;

// If k is 1, delete the head node

if (k == 1) {

ListNode* temp = head;

head = head->next;

delete temp;

return head;

}

// Initialize a temporary pointer

ListNode* temp = head;

// Traverse to the (k-1)th node

for (int i = 0; temp != NULL && i < k - 2; i++) {

temp = temp->next;

}

/*If k is greater than the number of nodes,

return the unchanged list*/

if (temp == NULL || temp->next == NULL)

```

        return head;

    // Delete the k-th node
    ListNode* next = temp->next->next;
    delete temp->next;
    temp->next = next;

    // Return head
    return head;
}

};

// Function to print the linked list
void printLL(ListNode* head) {
    ListNode* current = head;
    while (current != nullptr) {
        cout << current->val << " ";
        current = current->next;
    }
    cout << endl;
}

```

```
int main() {  
  
    // Initialize a vector with values for the linked list  
    vector<int> arr = {12, 5, 8, 7};  
  
    // Create a linked list with the values from the  
vector  
  
    ListNode* head = new ListNode(arr[0]);  
    head->next = new ListNode(arr[1]);  
    head->next->next = new ListNode(arr[2]);  
    head->next->next->next = new ListNode(arr[3]);  
  
    // Print the original linked list  
    cout << "Original list: ";  
    printLL(head);  
  
    // Creating an instance of Solution class  
    Solution sol;  
  
    // Call the deleteKthNode function to delete the  
k-th node  
  
    int k = 2;
```

```

head = sol.deleteKthNode(head, k);

// Print the linked list after deletion
cout << "List after deleting the kth node: ";
printLL(head);

return 0;
}

```

Delete the element with value X

```

#include <bits/stdc++.h>
using namespace std;

// Definition of singly linked list
class ListNode{
public:
int data;
ListNode *next;
ListNode() : data(0), next(nullptr) {}
ListNode(int x) : data(x), next(nullptr) {}
ListNode(int x, ListNode *next) : data(x),
next(next) {}
};

```


// Solution class

class Solution {

public:

**// To delete a node with a specific value in a
linked list**

**ListNode* deleteNodeWithValueX(ListNode*
&head, int X) {**

// Check if list is empty

if (head == NULL)

return head;

// If first node has target value, delete

if (head->data == X) {

ListNode* temp = head;

head = head->next;

delete temp;

return head;

}

ListNode* temp = head;

ListNode* prev = NULL;

```
/* Traverse the list to find  
the node with the target value*/  
while (temp != NULL) {  
    if (temp->data == X) {  
        // Adjust the pointers  
        prev->next = temp->next;  
        // Delete node  
        delete temp;  
        return head;  
    }  
    prev = temp;  
    temp = temp->next;  
}  
  
return head;  
}  
};
```

// Helper Function to print the linked list

```
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->data << " ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

int main() {

// Create a linked list from a vector

vector<int> arr = {0, 1, 2};

int X = 1;

ListNode* head = new ListNode(arr[0]);

head->next = new ListNode(arr[1]);

head->next->next = new ListNode(arr[2]);

// Print the original list

cout << "Original List: ";

printLL(head);

```
// Create a Solution object  
Solution sol;  
head = sol.deleteNodeWithValueX(head, X);  
  
// Print the modified linked list  
cout << "List after deleting the given value: ";  
printLL(head);  
  
return 0;  
}
```

Insertion at the head of Linked List

```
#include <bits/stdc++.h>  
using namespace std;  
  
// Definition of singly linked list  
struct ListNode  
{  
    int val;  
    ListNode *next;  
    ListNode(int data1)  
    {
```

```

    val = data1;
    next = NULL;
}
ListNode(int data1, ListNode *next1)
{
    val = data1;
    next = next1;
}
};

```

// Solution class

class Solution {

public:

// Function to insert at head

ListNode* insertAtHead(ListNode* head, int X) {

//Creating a new node

ListNode* newnode = new ListNode(X);

/* Making next of newly created node to

point the head of the LinkedList*/

newnode->next = head;

```
// Making newly created node as head  
head = newnode;  
  
// Return the head of modified list  
return head;  
}  
};
```

```
// Helper Function to print the linked list  
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

```
int main() {  
    // Create a linked list from a vector
```

```
vector<int> arr = {20, 30, 40};  
int X= 10;  
ListNode* head = new ListNode(arr[0]);  
head->next = new ListNode(arr[1]);  
head->next->next = new ListNode(arr[2]);  
  
// Print the original list  
cout << "Original List: ";  
printLL(head);  
  
// Create a Solution object  
Solution sol;  
head = sol.insertAtHead(head, X);  
  
// Print the modified linked list  
cout << "List after inserting the given value at  
head: ";  
printLL(head);  
  
return 0;  
}
```

Insertion at the tail of Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list
```

```
struct ListNode
```

```
{
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode(int data1)
```

```
{
```

```
        val = data1;
```

```
        next = NULL;
```

```
}
```

```
    ListNode(int data1, ListNode *next1)
```

```
{
```

```
        val = data1;
```

```
        next = next1;
```

```
}
```

```
};
```

```
// Solution class
```



```
class Solution {  
public:  
  
    // Function to insert a new node at the tail of the  
linked list  
  
    ListNode* insertAtTail(ListNode* &head, int X) {  
        if (head == NULL)  
            return new ListNode(X);  
  
        ListNode* temp = head;  
        // Traversing until the last node  
        while (temp->next != NULL) {  
            temp = temp->next;  
        }  
  
        ListNode* newNode = new ListNode(X);  
        temp->next = newNode;  
  
        return head;  
    }  
};  
  
// Helper Function to print the linked list
```

```
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

```
int main() {  
    // Create a linked list from a vector  
    vector<int> arr = {10, 20, 30};  
    int val = 40;  
    ListNode* head = new ListNode(arr[0]);  
    head->next = new ListNode(arr[1]);  
    head->next->next = new ListNode(arr[2]);  
  
    // Print the original list  
    cout << "Original List: ";  
    printLL(head);  
  
    // Create a Solution object
```

```
Solution sol;  
head = sol.insertAtTail(head, val);  
  
// Print the modified linked list  
cout << "List after inserting the given value at  
the tail:";  
printLL(head);  
  
return 0;  
}
```

Insertion at the Kth position of Linked List

```
#include <bits/stdc++.h>  
using namespace std;  
  
// Definition of singly linked list  
struct ListNode  
{  
    int val;  
    ListNode *next;  
    ListNode(int data1)  
    {  
        val = data1;
```

```

        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};

```

// Solution class

```
class Solution {
```

```
public:
```

```
    // Function to insert a new node at the kth position
```

```
    ListNode* insertAtKthPosition(ListNode* &head, int X, int K) {
```

```
        /* If the linked list is empty
```

```
        and k is 1, insert the
```

```
        new node as the head*/
```

```
        if (head == NULL) {
```

```
            if (K == 1)
```

```
                return new ListNode(X);
```

```
    else  
        return head;  
}
```

```
/*If K is 1, insert the new  
node at the beginning  
of the linked list*/
```

```
if (K == 1)  
    return new ListNode(X, head);
```

```
int cnt = 0;  
ListNode* temp = head;
```

```
/* Traverse the linked list  
to find the node at position k-1*/
```

```
while (temp != NULL) {  
    cnt++;
```

```
if (cnt == K-1) {
```

```
    /* Insert the new node after the node  
    at position k-1*/
```

```
    ListNode* newNode = new ListNode(X,  
temp->next);
```

```

        temp->next = newNode;
        break;
    }
    temp = temp->next;
}

return head;
}
};

// Helper Function to print the linked list
void printLL(ListNode* head) {
    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    // Create a linked list from a vector

```

```

vector<int> arr = {10, 30, 40};
int X = 20, K = 2;
ListNode* head = new ListNode(arr[0]);
head->next = new ListNode(arr[1]);
head->next->next = new ListNode(arr[2]);

// Print the original list
cout << "Original List: ";
printLL(head);

// Create a Solution object
Solution sol;
head = sol.insertAtKthPosition(head, X, K);

// Print the modified linked list
cout << "List after inserting the given value at
the Kth position: ";
printLL(head);

return 0;
}

```

Insertion before the value X in Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// ListNode definition
```

```
class ListNode{
```

```
public:
```

```
int data;
```

```
ListNode *next;
```

```
ListNode() : data(0), next(nullptr) {}
```

```
ListNode(int x) : data(x), next(nullptr) {}
```

```
ListNode(int x, ListNode *next) : data(x),  
next(next) {}
```

```
};
```

```
// Solution class
```

```
class Solution {
```

```
public:
```

```
// Function to insert a new node before the given  
node
```

```
ListNode* insertBeforeX(ListNode* &head, int X,  
int val) {
```

```
if (head == NULL) {
```



```

        return NULL;
    }

    /* Insert at the beginning if the
    value matches the head's data */
    if (head->data == X)
        return new ListNode(val, head);

    ListNode* temp = head;
    while (temp->next != NULL) {
        /* Insert at the current position if the
        next node has the desired value */
        if (temp->next->data == X) {
            ListNode* newNode = new ListNode(val,
temp->next);
            temp->next = newNode;
            break;
        }
        temp = temp->next;
    }
    return head;
}

```

```
};
```

```
// Helper Function to print the linked list
```

```
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->data << " ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

```
int main() {
```

```
    // Create a linked list from a vector
```

```
    vector<int> arr = {1, 2, 4, 5};
```

```
    int X = 4, val = 3;
```

```
    ListNode* head = new ListNode(arr[0]);
```

```
    head->next = new ListNode(arr[1]);
```

```
    head->next->next = new ListNode(arr[2]);
```

```
    // Print the original list
```

```
    cout << "Original List: ";
```

```
printLL(head);

// Create a Solution object
Solution sol;
head = sol.insertBeforeX(head, X, val);

// Print the modified linked list
cout << "List after inserting a new node before
the given node: ";
printLL(head);

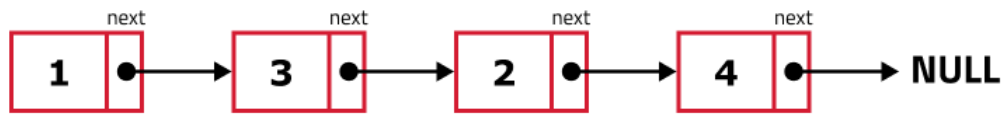
return 0;
}
```

Solution

Introduction to Doubly Linked Lists

A doubly linked list is an extension of a singly linked list. To fully grasp doubly linked lists, it is essential to understand the concepts of singly linked lists. Let's briefly recap the basics of singly linked lists.

Singly Linked Lists



A single linked list

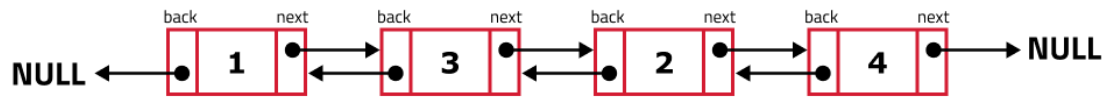
In a singly linked list, each node contains some data and a pointer to the next node in the sequence. The last node points to null, indicating the end of the list. This structure is linear and unidirectional; you can only traverse the list in one direction, from the first node to the last.

Here is a basic representation of a singly linked list: [Data] -> [Data] -> [Data] -> null

Each node stores:

- **Data:** The value of the node.
- **Next Pointer:** The address of the next node in the sequence.

Doubly Linked Lists



A doubly linked list

A doubly linked list improves upon the singly linked list by allowing traversal in both directions. Each node contains two pointers: one to the next node and one to the previous node. This bidirectional nature of the list offers greater flexibility in navigating and modifying the list.

Here is a basic representation of a doubly linked list: null <- [Data] <-> [Data] <-> [Data] -> null

Each node in a doubly linked list stores:

- **Data:** The value of the node.
- **Next Pointer:** The address of the next node.
- **Back Pointer:** The address of the previous node.

Advantages of Doubly Linked Lists:

- **Bidirectional Traversal:** You can traverse the list in both forward and backward directions, which can be helpful for certain operations.
- **More Efficient Deletion:** Deleting a node is more efficient because you have direct access

to the previous node, eliminating the need for traversal from the head to find the predecessor.

- Insertion Operations: Inserting a node before or after a given node is simpler because you have access to both neighboring nodes.**

Disadvantages:

- Increased Memory Usage: Each node requires extra memory for the backward pointer, which can be a drawback for memory-constrained environments.**
- Complexity: The implementation is more complex compared to singly linked lists due to the additional pointer and the need to maintain pointers in both directions.**

Representation in C++ and Java

In a doubly linked list, the node structure is extended to include a pointer to the previous node. Here's how you might define it in C++:

C++JavaPythonJavaScript

```
struct Node {
```

```
    // Value of the node
```

```
    int data;
```

```
    // Pointer to the next node
```

```

Node* next;

// Pointer to the previous node

Node* back;

// Constructor to initialize a node with data

Node(int data) : data(data), next(nullptr),
back(nullptr) {}

};

```

Conclusion:

In summary, a doubly linked list is a versatile data structure that allows for bidirectional traversal by adding a backward pointer to each node. This simple modification significantly enhances the flexibility of linked lists, making it easier to navigate in both directions.

17

Sudoku Solver - TUF+

Delete head of Doubly Linked List

```

#include <bits/stdc++.h>

using namespace std;

// Definition of doubly linked list

struct ListNode

```

```
{  
    int val;  
    ListNode *next;  
    ListNode *prev;  
    ListNode()  
    {  
        val = 0;  
        next = NULL;  
        prev = NULL;  
    }  
    ListNode(int data1)  
    {  
        val = data1;  
        next = NULL;  
        prev = NULL;  
    }  
    ListNode(int data1, ListNode *next1, ListNode  
*prev1)  
    {  
        val = data1;  
        next = next1;  
        prev = prev1;  
    }  
}
```



```
    }  
};
```

// Solution class

class Solution {

public:

**// Function to delete the head of the doubly
linked list**

ListNode* deleteHead(ListNode* head) {

if (head == nullptr || head->next == nullptr)

return nullptr; // Return NULL

// Store current head as 'prev'

ListNode* prev = head;

// Move 'head' to next node

head = head->next;

// Set 'prev' pointer

head->prev = nullptr;

// Set 'next' pointer

prev->next = nullptr;

```
    // Return new head  
    return head;  
}  
};
```

// Helper Function to convert an array to a doubly linked list

```
ListNode* arrayToLinkedList(vector<int> &nums) {  
    // If array is empty, return nullptr  
    if (nums.empty()) return nullptr;
```

// Create head node with first element of the array

```
    ListNode* head = new ListNode(nums[0]);
```

// Initialize 'prev' to the head node

```
    ListNode* prev = head;
```

```
    for (int i=1; i < nums.size(); i++) {
```

// Create a new node

```
        ListNode* temp = new ListNode(nums[i],  
        nullptr, prev);
```

```
// Update 'next' pointer  
prev->next = temp;  
  
// Move 'prev' to newly created node  
prev = temp;  
}  
  
  
// Return head  
return head;  
}  
  
  
// Helper Function to print the linked list  
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
    cout << endl;  
}  
  
  
int main() {  
    vector<int> nums = {1, 2, 3, 4, 5};
```

```
// Creating the doubly linked list from given array  
ListNode* head = arrayToLinkedList(nums);  
  
// Print the Original list  
cout << "Original List: ";  
printLL(head);  
  
// Create an instance of Solution class  
Solution sol;  
  
// Function call to delete the head of the doubly  
linked list  
head = sol.deleteHead(head);  
  
// Print the Modified list  
cout << "Modified list: ";  
printLL(head);  
  
return 0;  
}
```

Delete Tail of Doubly Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of doubly linked list
```

```
struct ListNode
```

```
{
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode *prev;
```

```
    ListNode()
```

```
    {
```

```
        val = 0;
```

```
        next = NULL;
```

```
        prev = NULL;
```

```
    }
```

```
    ListNode(int data1)
```

```
    {
```

```
        val = data1;
```

```
        next = NULL;
```

```
        prev = NULL;
```

```

    }

    ListNode(int data1, ListNode *next1, ListNode
*prev1)
    {
        val = data1;
        next = next1;
        prev = prev1;
    }
};

```

// Solution class

```
class Solution {
```

```
public:
```

```
// Function to delete the tail of a doubly linked
list
```

```
ListNode* deleteTail(ListNode* head) {
```

```
    if (head == nullptr || head->next == nullptr) {
```

```
        return nullptr;
```

```
    }
```

```
// Navigate to the tail of the linked list
```

```
ListNode* tail = head;
```

```
while (tail->next != nullptr)  
    tail = tail->next;  
  
// Update the pointers  
ListNode* newTail = tail->prev;  
newTail->next = nullptr;  
tail->prev = nullptr;  
  
// Free memory  
delete tail;  
  
return head; // Return head of modified list  
}  
};  
  
// Helper Function to convert an array to a doubly  
linked list  
ListNode* arrayToLinkedList(vector<int> &nums) {  
    // If array is empty, return nullptr  
    if (nums.empty()) return nullptr;
```

// Create head node with first element of the array

ListNode* head = new ListNode(nums[0]);

// Initialize 'prev' to the head node

ListNode* prev = head;

for (int i=1; i < nums.size(); i++) {

// Create a new node

**ListNode* temp = new ListNode(nums[i],
nullptr, prev);**

// Update 'next' pointer

prev->next = temp;

// Move 'prev' to newly created node

prev = temp;

}

// Return head

return head;

}

// Helper Function to print the linked list

void printLL(ListNode* head) {


```
while (head != NULL) {  
    cout << head->val << " ";  
    head = head->next;  
}  
cout << endl;  
}
```

```
int main() {  
    vector<int> nums = {1, 2, 3, 4, 5};  
  
    // Creating the doubly linked list from given array  
    ListNode* head = arrayToLinkedList(nums);  
  
    // Print the Original list  
    cout << "Original List: ";  
    printLL(head);  
  
    // Create an instance of Solution class  
    Solution sol;  
  
    // Function call to delete the tail of the doubly  
    linked list
```

```

head = sol.deleteTail(head);

// Print the Modified list
cout << "Modified list: ";
printLL(head);

return 0;
}

Delete Kth Element of Doubly Linked List
#include <bits/stdc++.h>
using namespace std;

// Definition of doubly linked list
struct ListNode
{
    int val;
    ListNode *next;
    ListNode *prev;
    ListNode()
    {
        val = 0;

```

```

        next = NULL;
        prev = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
        prev = NULL;
    }
    ListNode(int data1, ListNode *next1, ListNode
*prev1)
    {
        val = data1;
        next = next1;
        prev = prev1;
    }
};

```

// Solution class

class Solution {

public:

// Function to remove the Kth element

```
ListNode* deleteKthElement(ListNode* head, int  
k) {  
    // If the list is empty, return nullptr  
    if (head == nullptr)  
        return nullptr;  
  
    int count = 0;  
    ListNode* kNode = head;  
  
    // Traverse the list  
    while (kNode != nullptr) {  
        count++;  
        if (count == k) break;  
        kNode = kNode->next;  
    }  
  
    // If k is larger  
    if (kNode == nullptr) return head;  
  
    // Update the pointers  
    ListNode* prev = kNode->prev;  
    ListNode* front = kNode->next;
```

```
// If node to be deleted is only node in list  
if (prev == nullptr && front == nullptr) {  
    delete kNode;  
    return nullptr;  
}
```

```
// If node to be deleted is head of list  
else if (prev == nullptr) {  
    head = front;  
    front->prev = nullptr;  
}
```

```
// If node to be deleted is tail of list  
else if (front == nullptr) {  
    prev->next = nullptr;  
}
```

```
// If node to be deleted is in middle of list  
else {  
    prev->next = front;
```

```

        front->prev = prev;
    }

    // Free memory of the deleted node
    delete kNode;

    // Return modified list head
    return head;
}

};

// Helper Function to convert an array to a doubly
linked list

ListNode* arrayToLinkedList(vector<int> &nums) {
    // If array is empty, return nullptr
    if (nums.empty()) return nullptr;

    // Create head node with first element of the
    array

    ListNode* head = new ListNode(nums[0]);

    // Initialize 'prev' to the head node
    ListNode* prev = head;

```

```
for (int i=1; i < nums.size(); i++) {  
    // Create a new node  
    ListNode* temp = new ListNode(nums[i],  
nullptr, prev);  
    // Update 'next' pointer  
    prev->next = temp;  
    // Move 'prev' to newly created node  
    prev = temp;  
}  
  
// Return head  
return head;  
}  
  
// Helper Function to print the linked list  
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
    cout << endl;
```

}

int main() {

vector<int> nums = {1, 2, 3, 4, 5};

int k = 2;

// Creating the doubly linked list from given array

ListNode* head = arrayToLinkedList(nums);

// Print the Original list

cout << "Original List: ";

printLL(head);

// Create an instance of Solution class

Solution sol;

**// Function call to delete the kth Node of the
doubly linked list**

head = sol.deleteKthElement(head, k);

// Print the Modified list

cout << "Modified list: ";


```
printLL(head);  
  
return 0;  
}
```

Removing given node in Doubly Linked List

```
#include <bits/stdc++.h>  
  
using namespace std;  
  
// Definition of doubly linked list  
  
class ListNode {  
  
public:  
  
    int data;  
  
    ListNode *prev;  
  
    ListNode *next;  
  
    ListNode() : data(0), prev(nullptr), next(nullptr) {}  
  
    ListNode(int x) : data(x), prev(nullptr),  
next(nullptr) {}  
  
    ListNode(int x, ListNode *prev, ListNode *next) :  
data(x), prev(prev), next(next) {}  
  
};  
  
  
// Solution class
```

```
class Solution {  
public:  
  
    /* Function to delete the given node  
from doubly linked list */  
  
    void deleteGivenNode(ListNode* node) {  
  
        ListNode* prev = node->prev;  
  
        ListNode* front = node->next;  
  
  
        // Edge case if the given node is the tail node  
  
        if (front == nullptr) {  
  
            prev->next = nullptr;  
  
            node->prev = nullptr;  
  
            delete node;  
  
            return;  
  
        }  
  
  
        // Disconnect node  
  
        prev->next = front;  
  
        front->prev = prev;  
  
  
        // Set node's pointers to NULL
```

```
node->next = nullptr;  
node->prev = nullptr;  
  
// Free memory  
delete node;  
}  
};  
  
// Helper Function to convert an array to a doubly  
linked list  
  
ListNode* arrayToLinkedList(vector<int> &nums) {  
    // If array is empty, return nullptr  
    if (nums.empty()) return nullptr;  
  
    // Create head node with first element of the  
array  
  
    ListNode* head = new ListNode(nums[0]);  
  
    // Initialize 'prev' to the head node  
    ListNode* prev = head;  
  
    for (int i=1; i < nums.size(); i++) {  
        // Create a new node
```

```
ListNode* temp = new ListNode(nums[i], prev,  
nullptr);
```

```
// Update 'next' pointer
```

```
prev->next = temp;
```

```
// Move 'prev' to newly created node
```

```
prev = temp;
```

```
}
```

```
// Return head
```

```
return head;
```

```
}
```

```
// Helper Function to print the linked list
```

```
void printLL(ListNode* head) {
```

```
while (head != NULL) {
```

```
    cout << head->data << " ";
```

```
    head = head->next;
```

```
}
```

```
cout << endl;
```

```
}
```

```
int main() {
```

```
vector<int> nums = {1, 2, 3, 4, 5};
```

```
// Creating the doubly linked list from given array
```

```
ListNode* head = arrayToLinkedList(nums);
```

```
// Node to be deleted
```

```
ListNode* node = head-> next-> next;
```

```
// Print the Original list
```

```
cout << "Original List: ";
```

```
printLL(head);
```

```
// Create an instance of Solution class
```

```
Solution sol;
```

```
/* Function call to delete the given
```

```
node from the doubly linked list */
```

```
sol.deleteGivenNode(node);
```

```
// Print the Modified list
```

```
cout << "Modified list: ";
```

```
printLL(head);
```

```
    return 0;
}
```

Insert node before head in Doubly Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of doubly linked list
```

```
class ListNode {
```

```
public:
```

```
    int data;
```

```
    ListNode *prev;
```

```
    ListNode *next;
```

```
    ListNode() : data(0), prev(nullptr), next(nullptr) {}
```

```
    ListNode(int x) : data(x), prev(nullptr),  
next(nullptr) {}
```

```
    ListNode(int x, ListNode *prev, ListNode *next) :  
data(x), prev(prev), next(next) {}  
};
```

```
// Solution class
```

```
class Solution {
```

public:

```
/* Function to insert a node before  
head in a doubly linked list */  
ListNode* insertBeforeHead(ListNode* head, int  
X) {  
    // Create new node which will be the new head  
    ListNode* newHead = new ListNode(X, nullptr,  
head);  
  
    // Point the current head back to new one  
    head->prev = newHead;  
  
    return newHead; // Return new head  
}  
};
```

**// Helper Function to convert an array to a doubly
linked list**

```
ListNode* arrayToLinkedList(vector<int> &nums) {  
    // If array is empty, return nullptr  
    if (nums.empty()) return nullptr;
```

```
// Create head node with first element of the  
array  
ListNode* head = new ListNode(nums[0]);  
// Initialize 'prev' to the head node  
ListNode* prev = head;  
  
for (int i=1; i < nums.size(); i++) {  
    // Create a new node  
    ListNode* temp = new ListNode(nums[i], prev,  
nullptr);  
    // Update 'next' pointer  
    prev->next = temp;  
    // Move 'prev' to newly created node  
    prev = temp;  
}  
  
// Return head  
return head;  
}  
  
// Helper Function to print the linked list  
void printLL(ListNode* head) {
```



```
while (head != NULL) {  
    cout << head->data << " ";  
    head = head->next;  
}  
cout << endl;  
}
```

```
int main() {  
    vector<int> nums = {2, 3, 4, 5};  
  
    // Creating the doubly linked list from given array  
    ListNode* head = arrayToLinkedList(nums);  
  
    // Print the Original list  
    cout << "Original List: ";  
    printLL(head);  
  
    // Create an instance of Solution class  
    Solution sol;  
  
    /* Function call to insert a node before
```

```

head in a doubly linked list */

head = sol.insertBeforeHead(head, 1);


// Print the Modified list
cout << "Modified list: ";
printLL(head);


return 0;
}

Insert node before tail in Doubly Linked List
#include <bits/stdc++.h>
using namespace std;


// Definition of doubly linked list
class ListNode {
public:
int data;
ListNode *prev;
ListNode *next;
ListNode() : data(0), prev(nullptr), next(nullptr) {}
ListNode(int x) : data(x), prev(nullptr),
next(nullptr) {}

```

```
ListNode(int x, ListNode *prev, ListNode *next) :  
data(x), prev(prev), next(next) {}  
};
```

// Solution class

```
class Solution {
```

```
public:
```

```
// Function to insert a node before tail of a  
doubly linked list
```

```
ListNode* insertBeforeTail(ListNode* head, int  
X) {
```

```
// Edge case
```

```
if (head->next == nullptr) {
```

```
// Create new node with data as X
```

```
ListNode* newHead = new ListNode(X,  
nullptr, head);
```

```
head->prev = newHead;
```

```
return newHead;
```

```
}
```

```
// Create pointer tail
```

```
ListNode* tail = head;
```

```

while (tail->next != nullptr) {
    tail = tail->next;
}

// Keep track of node before tail using prev
ListNode* prev = tail->prev;

// Create new node with value X
ListNode* newNode = new ListNode(X, prev,
tail);

// Join the new node
prev->next = newNode;
tail->prev = newNode;

// Return updated linked list
return head;
}
};

// Helper Function to convert an array to a doubly
linked list
ListNode* arrayToLinkedList(vector<int> &nums) {

```

```
// If array is empty, return nullptr  
if (nums.empty()) return nullptr;  
  
// Create head node with first element of the  
array  
ListNode* head = new ListNode(nums[0]);  
  
// Initialize 'prev' to the head node  
ListNode* prev = head;  
  
for (int i=1; i < nums.size(); i++) {  
    // Create a new node  
    ListNode* temp = new ListNode(nums[i], prev,  
nullptr);  
    // Update 'next' pointer  
    prev->next = temp;  
    // Move 'prev' to newly created node  
    prev = temp;  
}  
  
// Return head  
return head;  
}
```

// Helper Function to print the linked list

```
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->data << " ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

int main() {

```
    vector<int> nums = {1, 2, 3, 5};
```

// Creating the doubly linked list from given array

```
    ListNode* head = arrayToLinkedList(nums);
```

// Print the Original list

```
    cout << "Original List: ";
```

```
    printLL(head);
```

// Create an instance of Solution class

Solution sol;

**/* Function call to insert a node before
head in a doubly linked list */**

head = sol.insertBeforeTail(head, 4);

// Print the Modified list

cout << "Modified list: ";

printLL(head);

return 0;

}

Insert node before (kth node) in Doubly Linked List

#include <bits/stdc++.h>

using namespace std;

// Definition of doubly linked list

class ListNode {

public:

int data;

ListNode *prev;

```

ListNode *next;

ListNode() : data(0), prev(nullptr), next(nullptr) {}

ListNode(int x) : data(x), prev(nullptr),
next(nullptr) {}

ListNode(int x, ListNode *prev, ListNode *next) :
data(x), prev(prev), next(next) {}

};

```

// Solution class

```

class Solution {

```

```

public:

```

```

    /* Function to insert a node before the

```

```

Kth node in a doubly linked list */

```

```

ListNode* insertBeforeKthPosition(ListNode*
head, int X, int K) {

```

```

    // If node has to be inserted before the head

```

```

if (K == 1) {

```

```

        ListNode* newHead = new ListNode(X,
nullptr, head);

```

```

        head->prev = newHead;

```

```

        return newHead;

```

```

}

```



```
// Temporary pointer  
ListNode* temp = head;  
  
// Reach kth node  
int count = 0;  
while (temp != nullptr) {  
    count++;  
  
    // If kth node is reached, Break out of the  
loop  
    if (count == K) break;  
  
    // Otherwise Keep moving temp forward  
    temp = temp->next;  
}  
  
// Track the node  
ListNode* prev = temp->prev;  
  
// Create new node with data as X
```

```
ListNode* newNode = new ListNode(X, prev,  
temp);
```

```
// Join new node
```

```
prev->next = newNode;
```

```
temp->prev = newNode;
```

```
// Return head
```

```
return head;
```

```
}
```

```
};
```

```
// Helper Function to convert an array to a doubly  
linked list
```

```
ListNode* arrayToLinkedList(vector<int> &nums) {
```

```
// If array is empty, return nullptr
```

```
if (nums.empty()) return nullptr;
```

```
// Create head node with first element of the  
array
```

```
ListNode* head = new ListNode(nums[0]);
```

```
// Initialize 'prev' to the head node
```

```
ListNode* prev = head;

for (int i=1; i < nums.size(); i++) {
    // Create a new node

    ListNode* temp = new ListNode(nums[i], prev,
nullptr);

    // Update 'next' pointer

    prev->next = temp;

    // Move 'prev' to newly created node

    prev = temp;
}

// Return head

return head;
}

// Helper Function to print the linked list
void printLL(ListNode* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
```

```
cout << endl;  
}
```

```
int main() {  
    vector<int> nums = {1, 2, 3, 5};  
  
    // Creating the doubly linked list from given array  
    ListNode* head = arrayToLinkedList(nums);  
  
    // Print the Original list  
    cout << "Original List: ";  
    printLL(head);  
  
    // Create an instance of Solution class  
    Solution sol;  
  
    /* Function call to insert a node before the  
    Kth node in a doubly linked list */  
    head = sol.insertBeforeKthPosition(head, 4, 4);  
  
    // Print the Modified list
```

```
cout << "Modified list: ";  
printLL(head);  
  
return 0;  
}
```

Insert before given node in Doubly Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of doubly linked list
```

```
class ListNode {
```

```
public:
```

```
int data;
```

```
ListNode *prev;
```

```
ListNode *next;
```

```
ListNode() : data(0), prev(nullptr), next(nullptr) {}
```

```
ListNode(int x) : data(x), prev(nullptr),  
next(nullptr) {}
```

```
ListNode(int x, ListNode *prev, ListNode *next) :  
data(x), prev(prev), next(next) {}
```

```
};
```

// Solution class

class Solution {

public:

**/* Function to insert a new node before
given node in a doubly linked list */**

**void insertBeforeGivenNode(ListNode* node, int
X) {**

// Get node before the given node

ListNode* prev = node->prev;

// Create new node

**ListNode* newNode = new ListNode(X, prev,
node);**

// Connect newNode

prev->next = newNode;

node->prev = newNode;

// void function to just update

return;

}

};

// Helper Function to convert an array to a doubly linked list

ListNode* arrayToLinkedList(vector<int> &nums) {

// If array is empty, return nullptr

if (nums.empty()) return nullptr;

// Create head node with first element of the array

ListNode* head = new ListNode(nums[0]);

// Initialize 'prev' to the head node

ListNode* prev = head;

for (int i=1; i < nums.size(); i++) {

// Create a new node

ListNode* temp = new ListNode(nums[i], prev, nullptr);

// Update 'next' pointer

prev->next = temp;

// Move 'prev' to newly created node

prev = temp;

}

```
// Return head  
return head;  
}
```

// Helper Function to print the linked list

```
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->data << " ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

```
int main() {
```

```
    vector<int> nums = {1, 2, 4, 5};
```

// Creating the doubly linked list from given array

```
ListNode* head = arrayToLinkedList(nums);
```

// Node before which the new node must be inserted


```
ListNode* node = head-> next-> next;
```

```
// Print the Original list
```

```
cout << "Original List: ";
```

```
printLL(head);
```

```
// Create an instance of Solution class
```

```
Solution sol;
```

```
/* Function call to insert a new node before  
given node in a doubly linked list */
```

```
sol.insertBeforeGivenNode(node, 3);
```

```
// Print the Modified list
```

```
cout << "Modified list: ";
```

```
printLL(head);
```

```
return 0;
```

```
}
```

Add two numbers in Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of Singly Linked List
```

```
struct ListNode {
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode() {
```

```
        val = 0;
```

```
        next = NULL;
```

```
    }
```

```
    ListNode(int data1){
```

```
        val = data1;
```

```
        next = NULL;
```

```
    }
```

```
    ListNode(int data1, ListNode *next1){
```

```
        val = data1;
```

```
        next = next1;
```

```
    }
```

```
};
```

```
class Solution {
```

public:

```
//Function to add two numbers as linked list  
  
ListNode* addTwoNumbers(ListNode* l1,  
ListNode* l2) {  
  
    /*Dummy node to act as the  
    starting point of the result list*/  
  
    ListNode *dummy = new ListNode();  
  
    /*Temp pointer to build  
    the result list*/  
  
    ListNode *temp = dummy;  
  
    //Initialize carry  
  
    int carry = 0;  
  
  
    /*Iterate while there are nodes in l1 or l2,  
    or there's a carry to process*/  
  
    while ((l1 != NULL || l2 != NULL) || carry) {  
        int sum = 0;  
  
  
        /*Add the value from l1  
        if available*/  
  
        if (l1 != NULL) {  
            sum += l1->val;
```

```
    l1 = l1->next;  
}
```

```
/*Add the value from l2
```

```
if available*/
```

```
if (l2 != NULL) {
```

```
    sum += l2->val;
```

```
    l2 = l2->next;
```

```
}
```

```
//Add the carry
```

```
sum += carry;
```

```
//Update the carry
```

```
carry = sum / 10;
```

```
/* Create a new node with the digit value
```

```
and attach it to the result list*/
```

```
ListNode *node = new ListNode(sum % 10);
```

```
temp->next = node;
```

```
/*Move to the
```

```
next position in the result list*/
```

```
        temp = temp->next;
    }
    /*Return the result list
    skipping the dummy node*/
    return dummy->next;
}
};
```

// Function to print the linked list

```
void printList(ListNode* head) {
    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
    }
    cout << endl;
}
```

int main() {

//Manual creation of linked list

```
    ListNode* l1 = new ListNode(2, new ListNode(4,
new ListNode(3)));
```

```
ListNode* l2 = new ListNode(5, new ListNode(6,  
new ListNode(4)));
```

```
//Instance of solution class
```

```
Solution solution;
```

```
ListNode* result = solution.addTwoNumbers(l1,  
l2);
```

```
// Print the result
```

```
printList(result);
```

```
return 0;
```

```
}
```

Segregate odd and even nodes in Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list
```

```
class ListNode{
```

```
public:
```

```
int data;
```

```
ListNode *next;
```

```
ListNode() : data(0), next(nullptr) {}  
ListNode(int x) : data(x), next(nullptr) {}  
ListNode(int x, ListNode *next) : data(x),  
next(next) {}  
};
```

```
class Solution {
```

```
public:
```

```
// Function to segregate odd and even indices  
nodes
```

```
ListNode* oddEvenList(ListNode* head) {
```

```
// Check if list is empty or has only one node
```

```
if (head == NULL || head->next == NULL)
```

```
    return head;
```

```
// To store values
```

```
vector<int> array;
```

```
ListNode* temp = head;
```

```
/*Traverse the list, skipping one node,
```

```
and store values in the vector*/
```

```
while (temp != NULL && temp->next != NULL) {
```

```
array.push_back(temp->data);  
temp = temp->next->next;  
}
```

```
/* If the traversal ends on a valid  
odd-indexed node, include its value as well*/
```

```
if (temp != NULL)  
    array.push_back(temp->data);
```

```
// Reset temp  
temp = head->next;
```

```
/* Traverse the list again, skipping one node  
and store values
```

```
in the vector*/
```

```
while (temp != NULL && temp->next != NULL) {  
    array.push_back(temp->data);  
    temp = temp->next->next;  
}
```

```
/* If the traversal ends on a valid
```



```
even-indexed node, include its value as well */  
if (temp != NULL)  
    array.push_back(temp->data);  
  
// Reset temp  
temp = head;  
int i = 0;  
  
// Update node values  
while (temp != NULL) {  
    temp->data = array[i];  
    temp = temp->next;  
    i++;  
}  
  
return head;  
}  
};
```

```
// Function to print the linked list  
void printLL(ListNode* head) {
```

```

while (head != NULL) {
    cout << head->data << " ";
    head = head->next;
}
cout << endl;
}

// Main function
int main() {
    // Create a linked list with given values
    vector<int> arr = {1, 3, 4, 2, 5, 6};
    ListNode* head = new ListNode(arr[0]);
    head->next = new ListNode(arr[1]);
    head->next->next = new ListNode(arr[2]);
    head->next->next->next = new ListNode(arr[3]);
    head->next->next->next->next = new
ListNode(arr[4]);
    head->next->next->next->next->next = new
ListNode(arr[5]);

    // Rearrange the list and print it
    Solution solution;

```

```

    head = solution.oddEvenList(head);
    printLL(head);

    return 0;
}

#include <bits/stdc++.h>
using namespace std;

// Definition of singly linked list
class ListNode{
public:
    int data;
    ListNode *next;
    ListNode() : data(0), next(nullptr) {}
    ListNode(int x) : data(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : data(x),
next(next) {}
};

class Solution {
public:
    // Function to rearrange nodes

```

```
ListNode* oddEvenList(ListNode* head) {  
    if (!head || !head->next)  
        return head;  
  
    /* Initialize pointers for odd  
    and even nodes and keep  
    track of the first even node*/  
  
    ListNode* odd = head;  
    ListNode* even = head->next;  
    ListNode* firstEven = head->next;  
  
// Rearranging nodes  
while (even && even->next) {  
    odd->next = odd->next->next;  
    even->next = even->next->next;  
    odd = odd->next;  
    even = even->next;  
}  
  
/* Connect the last odd  
node to the first even node*/
```

```
    odd->next = firstEven;

    return head;
}
};
```

// Function to print the linked list

```
void printLL(ListNode* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
}
```

// Main function

```
int main() {
    // Create a linked list with given values
    vector<int> arr = {1, 3, 4, 2, 5, 6};
    ListNode* head = new ListNode(arr[0]);
    head->next = new ListNode(arr[1]);
```

```
head->next->next = new ListNode(arr[2]);  
head->next->next->next = new ListNode(arr[3]);  
head->next->next->next->next = new  
ListNode(arr[4]);  
head->next->next->next->next->next = new  
ListNode(arr[5]);
```

```
// Rearrange the list and print it
```

```
Solution solution;
```

```
head = solution.oddEvenList(head);
```

```
printLL(head);
```

```
return 0;
```

```
}
```

Sort a Linked List of 0's 1's and 2's

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list:
```

```
class ListNode{
```

```
public:
```

```
int data;  
ListNode *next;  
ListNode() : data(0), next(nullptr) {}  
ListNode(int x) : data(x), next(nullptr) {}  
ListNode(int x, ListNode *next) : data(x),  
next(next) {}  
};
```

```
class Solution {  
public:  
  
// Function to sort the linked list  
ListNode* sortList(ListNode* head) {  
  
// Initialize counts  
  
int c0 = 0, c1 = 0, c2 = 0;  
  
ListNode* temp = head;  
  
  
/* Count the number of 0s,  
1s, and 2s in the list*/  
while (temp != NULL) {  
  
if (temp->data == 0)  
    c0++;  
  
else if (temp->data == 1)
```

```
        c1++;  
else if (temp->data == 2)  
        c2++;  
        temp = temp->next;  
}
```

```
temp = head;
```

```
/* Reassign values to  
the nodes based on  
the counts*/  
while (temp != NULL) {  
    if (c0 > 0) {  
        temp->data = 0;  
        c0--;  
    } else if (c1 > 0) {  
        temp->data = 1;  
        c1--;  
    } else if (c2 > 0) {  
        temp->data = 2;  
        c2--;
```



```
    }  
    temp = temp->next;  
}  
  
return head;  
}  
};
```

// Function to print linked list

```
void printList(ListNode* head) {  
    while (head != NULL) {  
        cout << head->data << " ";  
        head = head->next;  
    }  
    cout << endl;  
}
```

// Function to create new node

```
ListNode* newNode(int data) {  
    ListNode* node = new ListNode(data);  
    return node;  
}
```

}

int main() {

// Creating a linked list

ListNode* head = newNode(1);

head->next = newNode(2);

head->next->next = newNode(0);

head->next->next->next = newNode(1);

head->next->next->next->next = newNode(2);

**head->next->next->next->next->next =
newNode(0);**

**head->next->next->next->next->next->next =
newNode(1);**

// Print original list

cout << "Original list: ";

printList(head);

// Sort the list

Solution sol;

head = sol.sortList(head);

```

// Print sorted list

cout << "Sorted list: ";

printList(head);


return 0;
}

#include <bits/stdc++.h>


using namespace std;


// Definition of singly linked list:

class ListNode{
public:
int data;
ListNode *next;
ListNode() : data(0), next(nullptr) {}
ListNode(int x) : data(x), next(nullptr) {}
ListNode(int x, ListNode *next) : data(x),
next(next) {}
};


class Solution {

```

public:

// Function to sort the linked list

ListNode* sortList(ListNode* head) {

**/* If the list is empty or has only one
node, return as it is already sorted*/**

if (head == NULL || head->next == NULL)

return head;

// Dummy nodes to point to heads of

// three lists

ListNode* zeroHead = new ListNode(-1);

ListNode* oneHead = new ListNode(-1);

ListNode* twoHead = new ListNode(-1);

// Pointers to current last nodes of

// three lists

ListNode* zero = zeroHead;

ListNode* one = oneHead;

ListNode* two = twoHead;

ListNode* temp = head;

```
/* Traverse the original list  
and distribute the nodes  
into three lists*/
```

```
while (temp != NULL) {  
    if (temp->data == 0) {  
        zero->next = temp;  
        zero = temp;  
    } else if (temp->data == 1) {  
        one->next = temp;  
        one = temp;  
    } else if (temp->data == 2) {  
        two->next = temp;  
        two = temp;  
    }  
    temp = temp->next;  
}
```

```
// Connect the three lists together  
zero->next = (oneHead->next) ? oneHead-  
>next : twoHead->next;  
one->next = twoHead->next;  
two->next = NULL;
```

```

// New head of the sorted list

ListNode* newHead = zeroHead->next;


// Delete dummy nodes

delete zeroHead;

delete oneHead;

delete twoHead;


return newHead;
}
};


// Function to print linked list
void printList(ListNode* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
}

```

// Function to create new node

```
ListNode* newNode(int data) {  
    ListNode* node = new ListNode(data);  
    return node;  
}
```

int main() {

// Creating a linked list

```
ListNode* head = newNode(1);  
head->next = newNode(2);  
head->next->next = newNode(0);  
head->next->next->next = newNode(1);  
head->next->next->next->next = newNode(2);  
head->next->next->next->next->next =  
newNode(0);  
head->next->next->next->next->next->next =  
newNode(1);
```

// Print original list

```
cout << "Original list: ";  
printList(head);
```

```

// Sort the list

Solution sol;

head = sol.sortList(head);


// Print sorted list

cout << "Sorted list: ";
printList(head);


return 0;
}

Remove Nth node from the back of the LL

#include<bits/stdc++.h>

using namespace std;

//Definition of singly linked list:

struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {

```



```

    val = 0;
    next = NULL;
}
ListNode(int data1)
{
    val = data1;
    next = NULL;
}
ListNode(int data1, ListNode *next1)
{
    val = data1;
    next = next1;
}
};

class Solution {
public:
    //Function to remove the nth node from end
    ListNode* removeNthFromEnd(ListNode* head,
int n) {
        if (head == NULL) {
            return NULL;

```

```
}
```

```
int cnt = 0;
```

```
ListNode* temp = head;
```

```
// Count the number of nodes
```

```
while (temp != NULL) {
```

```
    cnt++;
```

```
    temp = temp->next;
```

```
}
```

```
/*If N equals
```

```
the total number of nodes
```

```
delete the head*/
```

```
if (cnt == n) {
```

```
    ListNode* newhead = head->next;
```

```
    delete (head);
```

```
    return newhead;
```

```
}
```

```
/* Calculate the position
```

```
of the node to delete (res)*/
```

```
int res = cnt - n;  
temp = head;  
  
/* Traverse to the node  
just before the one to delete*/  
while (temp != NULL) {  
    res--;  
    if (res == 0) {  
        break;  
    }  
    temp = temp->next;  
}  
  
//Delete the Nth node from the end  
ListNode* delNode = temp->next;  
temp->next = temp->next->next;  
delete (delNode);  
return head;  
}  
};
```

//Function to print the linked list

```
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
}
```

```
int main() {  
    vector<int> arr = {1, 2, 3, 4, 5};  
    int N = 3;  
    ListNode* head = new ListNode(arr[0]);  
    head->next = new ListNode(arr[1]);  
    head->next->next = new ListNode(arr[2]);  
    head->next->next->next = new ListNode(arr[3]);  
    head->next->next->next->next = new  
    ListNode(arr[4]);
```

//Solution instance

Solution sol;

head = sol.removeNthFromEnd(head, N);

printLL(head);

```
        return 0;
    }

#include<bits/stdc++.h>
using namespace std;

//Definition of
//Single Linked List
struct ListNode
{
    int val;
    ListNode *next;
    ListNode()
    {
        val = 0;
        next = NULL;
    }
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
}
```

```
ListNode(int data1, ListNode *next1)
{
    val = data1;
    next = next1;
}
};

class Solution {
public:

    //Function to remove the nth node from end

    ListNode* removeNthFromEnd(ListNode* head,
int n) {

        //Creating pointers

        ListNode* fastp = head;
        ListNode* slowp = head;

        /*Move the fastp pointer
N nodes ahead*/

        for (int i = 0; i < n; i++) {
            fastp = fastp->next;
}
```

```

/*If fastp becomes NULL
the Nth node from the
end is the head*/
if (fastp == NULL) {
    return head->next;
}

/*Move both pointers
Until fastp reaches the end*/
while (fastp->next != NULL) {
    fastp = fastp->next;
    slowp = slowp->next;
}

//Delete the Nth node from the end
ListNode* delNode = slowp->next;
slowp->next = slowp->next->next;
delete delNode;
return head;
}
};

```

//Function to print the linked list

void printLL(ListNode* head) {

while (head != NULL) {

cout << head->val << " ";

head = head->next;

}

}

int main() {

vector<int> arr = {1, 2, 3, 4, 5};

int N = 3;

//Creation of linked list

ListNode* head = new ListNode(arr[0]);

head->next = new ListNode(arr[1]);

head->next->next = new ListNode(arr[2]);

head->next->next->next = new ListNode(arr[3]);

**head->next->next->next->next = new
ListNode(arr[4]);**

// Solution instance

Solution sol;


```
head = sol.removeNthFromEnd(head, N);  
printLL(head);  
  
return 0;  
}
```

Reverse a LL

```
#include <bits/stdc++.h>  
using namespace std;  
  
//Definition of singly linked list:  
struct ListNode  
{  
    int val;  
    ListNode *next;  
    ListNode()  
    {  
        val = 0;  
        next = NULL;  
    }
```

```

ListNode(int data1)
{
    val = data1;
    next = NULL;
}

ListNode(int data1, ListNode *next1)
{
    val = data1;
    next = next1;
}

};

class Solution {
public:

    /*Function to reverse a linked list
Using the 3-pointer approach*/
ListNode* reverseList(ListNode* head) {
    /*Initialize 'temp' at
head of linked list*/
    ListNode* temp = head;

```

**/* Initialize pointer 'prev' to NULL,
representing the previous node*/**

ListNode* prev = NULL;

**/* Traverse the list, continue till
'temp' reaches the end (NULL)*/**

while(temp != NULL){

/* Store the next node in

'front' to preserve the reference*/

ListNode* front = temp->next;

**/* Reverse the direction of the
current node's 'next' pointer
to point to 'prev'*/**

temp->next = prev;

**/* Move 'prev' to the current
node for the next iteration*/**

prev = temp;

/* Move 'temp' to the 'front' node

```

        advancing the traversal*/
        temp = front;
    }

    /*Return the new head of
    the reversed linked list*/
    return prev;
}

};

// Function to print the linked list
void printLinkedList(ListNode* head) {
    ListNode* temp = head;
    while (temp != nullptr) {
        cout << temp->val << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {

```

```
// Create a linked list with  
// Values 1, 3, 2, and 4  
ListNode* head = new ListNode(1);  
head->next = new ListNode(3);  
head->next->next = new ListNode(2);  
head->next->next->next = new ListNode(4);  
  
// Print the original linked list  
cout << "Original Linked List: ";  
printLinkedList(head);  
  
// Solution instance  
Solution sol;  
// Reverse the linked list  
head = sol.reverseList(head);  
  
// Print the reversed linked list  
cout << "Reversed Linked List: ";  
printLinkedList(head);  
  
return 0;
```

```
}
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list:
```

```
struct ListNode
```

```
{
```

```
    int val;
```

```
    ListNode *next;
```

```
    ListNode()
```

```
    {
```

```
        val = 0;
```

```
        next = NULL;
```

```
    }
```

```
    ListNode(int data1)
```

```
    {
```

```
        val = data1;
```

```
        next = NULL;
```

```
    }
```

```
    ListNode(int data1, ListNode *next1)
```

```
    {
```

```
    val = data1;
    next = next1;
}
};
```

```
class Solution {
```

```
public:
```

```
    /* Function to reverse a singly linked list using  
    recursion */
```

```
    ListNode* reverseList(ListNode* head) {
```

```
        /* Base case:
```

```
        If the linked list is empty or has only one  
        node,
```

```
        return the head as it is already reversed. */
```

```
        if (head == NULL || head->next == NULL) {
```

```
            return head;
```

```
        }
```

```
        /* Recursive step:
```

```
        Reverse the linked list starting
```

```
        from the second node (head->next). */
```

```
ListNode* newHead = reverseList(head->next);
```

```
/* Save a reference to the node following  
the current 'head' node. */
```

```
ListNode* front = head->next;
```

```
/* Make the 'front' node point  
to the current  
'head' node in the  
reversed order. */
```

```
front->next = head;
```

```
/* Break the link from  
the current 'head' node  
to the 'front' node  
to avoid cycles. */
```

```
head->next = NULL;
```

```
/* Return the 'newHead,'  
which is the new  
head of the reversed
```



```
        linked list. */  
        return newHead;  
    }  
};
```

// Function to print the linked list

```
void printLinkedList(ListNode* head) {  
    ListNode* temp = head;  
    while (temp != nullptr) {  
        cout << temp->val << " ";  
        temp = temp->next;  
    }  
    cout << endl;  
}
```

```
int main() {
```

// Create a linked list with values 1, 3, 2, and 4

```
ListNode* head = new ListNode(1);  
head->next = new ListNode(3);  
head->next->next = new ListNode(2);  
head->next->next->next = new ListNode(4);
```

```
// Print the original linked list  
cout << "Original Linked List: ";  
printLinkedList(head);  
  
// Solution instance  
Solution sol;  
  
// Reverse the linked list  
head = sol.reverseList(head);  
  
// Print the reversed linked list  
cout << "Reversed Linked List: ";  
printLinkedList(head);  
  
return 0;  
}
```